Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

PAPERCLIP

AI AGENT COMPANY OS

Manage AI Agents with Paperclip (TGS-2023036646)

TGS Ref No: TGS-2023036646

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	28 August 2026	Initial release — Learner Guide for Manage AI Agents with Paperclip (TGS-2023036646): four topics, six hands-on labs, each building a different simulated one-person AI agent company on synthetic data.	Dr. Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Introduction

This Learner Guide accompanies the WSQ course Manage AI Agents with Paperclip (TGS-2023036646), conducted by Tertiary Infotech Academy Pte Ltd. It gives the full step-by-step procedure for all six hands-on labs, organised by the four course topics. Each lab builds a DIFFERENT simulated one-person AI agent company — short-video production, digital marketing, finance, accounting, stock-trading research and HR people operations — so you see the same control model applied across six very different businesses.

Paperclip is an organisational control plane for AI agents: an open-source Node.js server with a React UI that gives a team of agents an org chart, issues, budgets, approval gates and a durable audit trail. It does not replace your agent runtime. The runtimes — Claude Code, Codex, Cursor, Gemini CLI, a bash process or an HTTP bot — still do the reasoning and the tool use; Paperclip schedules, budgets and audits them. Keeping that boundary clear is the single most useful idea in this course.

Use this guide alongside the course slides and the lab folders in labs/ of the course repository. Every data set in this course is SYNTHETIC and fabricated for training. Never load real company, market, financial or candidate data into a practice instance, and never place a real credential in a prompt, a lab note or Git.

Course Learning Outcomes

- LO1: Identify agentic AI opportunities and perform feasibility scans to determine when a Paperclip-managed multi-agent company is the appropriate solution.
- LO2: Utilise Paperclip to integrate agent runtimes, data and skills, and develop a goal-aligned AI agent company with an org chart, workspaces and budgets.
- LO3: Perform tests in Paperclip to verify that agents, issues, hand-offs and routines function correctly between disparate components.
- LO4: Resolve technical issues in Paperclip and implement modifications to the agent company, its governance controls and its cost boundaries.

Before You Start — Environment Setup

What you need

- Node.js 20 or later, and npm/npx. Paperclip will not onboard on an older runtime — check with `node --version` first.
- A NORMAL, non-elevated shell. Close any Administrator or root terminal before you begin: the embedded PostgreSQL refuses to start under an elevated user.
- One trainer-approved agent runtime (for example Claude Code, Codex or Gemini CLI), already installed AND authenticated OUTSIDE Paperclip.
- Git, used to baseline each agent workspace so that every later change is a readable, attributable diff.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Permission to create a local practice folder and to download the Paperclip npm package.
- The course lab repository, cloned from the link on your LMS course page.

Install and verify Paperclip

Onboarding downloads Paperclip, writes the local configuration, initialises the embedded PostgreSQL database, starts the server on port 3100 and normally opens your browser. Run it from a normal user shell — never with sudo. Then prove the control plane is healthy BEFORE you create any company: the diagnostic and the health endpoint together are your clean baseline, and every later adapter or budget failure is far easier to isolate because you recorded one.

```
node --version                # must report v20 or later
npm --version
npx --registry https://registry.npmjs.org paperclipai onboard --yes
npx paperclipai doctor        # fix the EARLIEST error first
curl -sS http://localhost:3100/api/health   # expect status ok
```

Conventions used in every lab

- Commands are run from a normal user terminal in the lab's practice root. A leading sudo is never required.
- Each lab uses its own simulated company and its own synthetic data set, supplied in that lab's data/ folder.
- Each agent gets a DEDICATED workspace folder; artifacts must land in the right one, and a marker in the wrong root is a boundary failure.
- Every issue is created UNASSIGNED — assignment is the trigger that wakes an agent.
- Bound every wait (about 10 minutes). An unbounded wait hides a failed adapter behind apparent patience.
- Record evidence as you go in evidence/lab0N-*.md, and never copy a key, token, cookie or connection string into it.

Topic 01 — Fundamentals of Agentic AI and Paperclip (25%)

Agentic AI foundations · the control-plane model · the twelve subsystems · feasibility and when NOT to use multi-agent

Key concepts

- Agentic AI adds reasoning, tool use, memory and autonomy on top of a language model, so the system completes work rather than only producing text.
- Paperclip is an organisational control plane — a Node.js server with a React UI that coordinates agents, goals, work, cost and governance. It is not an agent framework and it does not perform the reasoning itself.
- The defining separation: Paperclip manages the company; adapters run the agents. Organisational authority, runtime capability and verification evidence stay apart.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- A company is the boundary for one mission, its human members, AI employees, projects, issues, budget and activity history. One deployment holds many companies.
- The four pillars are the agentic task manager, the org chart for agents, agent employee training (skills and evaluations) and the agentic OS (runtime, sandboxing, MCP, RBAC and cost control).
- Twelve subsystems make up the platform: identity, org chart, work, heartbeat, workspaces, governance, budget, routines, plugins, secrets, activity and portability.
- A heartbeat is a bounded execution burst triggered by a timer, an assignment, an on-demand wake or a routine. Between heartbeats the agent is dormant, not idling.
- Work is checked out atomically with an execution lock, so two heartbeats can never double-work the same issue or race past a budget ceiling.
- 'If it can receive a heartbeat, it's hired' — adapters exist for Claude Code, Codex, Cursor, Gemini CLI, bash processes and HTTP/webhook bots.
- Feasibility comes before architecture: use one agent when the inputs, tools, authority and verification are the same across the whole task.
- Add a specialist only when different expertise is genuinely required, or when evidence must be produced independently of the agent that will use it.
- Installation is ready only when Node.js, the Paperclip configuration, the embedded database, server health, the browser UI and the doctor diagnostic all agree.

Lab 1 — Install Paperclip and Prove the Control Plane (ClipReel Studio)

Maps to: LO1: Explain the control-plane / runtime boundary and verify a healthy local Paperclip installation with observable evidence.

Simulated company: ClipReel Studio (Short Video Production).

Goal: You prepare a dedicated synthetic practice root for ClipReel Studio, a one-person short-video production company, verify the supported Node.js runtime, onboard Paperclip as a normal (non-elevated) user, and then inspect BOTH the built-in diagnostic and the live health endpoint. Proving the control plane before you hire any agent matters because every later adapter, budget or heartbeat failure is far easier to isolate when the platform itself has a recorded clean baseline.

What you'll build

A running Paperclip instance at http://localhost:3100 plus evidence/lab01-installation.md recording the Node/npm versions, the doctor summary and the /api/health response — with no credentials captured. (Tools: Node.js 20+, npm/npx, Paperclip CLI, browser, terminal or PowerShell.)

Step-by-step

1. Create the ClipReel practice root with separate workspace and evidence folders. Keep this root for the whole course.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
mkdir -p paperclip-tgs/clipreel/workspace/{head-of-content,trend-researcher,script-writer} \
         paperclip-tgs/clipreel/evidence
cd paperclip-tgs/clipreel
```

2. Verify the runtime. Node.js must report major version 20 or later. Record both lines in your evidence note — a version mismatch is the single most common cause of a failed onboard.

```
node --version
npm --version
```

3. Run the official onboarding command from a NORMAL user shell. The first run downloads Paperclip, writes the configuration, initialises the embedded PostgreSQL, starts the server and opens the browser. Do not prepend sudo.

```
npx --registry https://registry.npmjs.org paperclipai onboard --yes
```

4. If onboarding is still attached to the terminal, leave it running and open a second terminal in the same folder. If you stopped it, start it again.

```
npx paperclipai run
```

5. Run the built-in diagnostic. Fix the EARLIEST reported dependency or configuration error before continuing — never change several settings at once.

```
npx paperclipai doctor
```

6. Query the health endpoint and confirm it reports an ok status. This is the control plane answering for itself.

```
# macOS / Linux
curl -sS http://localhost:3100/api/health


# Windows PowerShell
Invoke-RestMethod -Uri 'http://localhost:3100/api/health' | ConvertTo-Json -Depth 8
```

7. Open the dashboard in a browser and confirm the Paperclip start or onboarding screen loads without a connection error. Do NOT create the company yet — that is Lab 2.

```
http://localhost:3100
```

8. Copy the ClipReel mock brief into the practice root so later labs have a realistic, synthetic mission to work from.

```
cp labs/lab-01-clipreel-studio/data/clipreel-company-brief.md ./
```

9. Write evidence/lab01-installation.md recording the date, Node and npm versions, the Paperclip version if shown, the doctor summary, the health status and the dashboard URL. Do NOT copy environment values, connection strings, cookies, tokens or provider keys.

```
evidence/lab01-installation.md
```

Test it

`npx paperclipai doctor` completes with no unresolved blocking error, GET http://localhost:3100/api/health returns status ok, the browser loads the Paperclip UI, and evidence/lab01-installation.md records all three checks with no credential or personal identifier.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: The lab folder labs/lab-01-* holds this lab's README and its synthetic data set. All data is fabricated for training — never substitute real company, market, financial or candidate data, and never record a credential in your evidence.

Topic 02 — Building AI Agents with Paperclip (25%)

Install and verify · company and mission · the CEO · org chart · runtime adapters · workspaces, instructions and secrets

Key concepts

- Onboarding with `paperclipai onboard --yes` downloads Paperclip, writes the configuration, initialises the embedded PostgreSQL and serves the UI on port 3100.
- Bind mode is a security decision: the default trusted loopback is the least exposed; `--bind lan` and `--bind tailnet` widen reach and require authentication.
- The onboarding wizard captures the company name, the mission statement and the team lead, then verifies connectivity to your chosen local agent runtime.
- A mission must be measurable. 'Grow the channel' cannot be verified; 'publish 8 short videos per month at ≥5% average watch-through' can.
- The CEO is the first hire and the top of the org chart — it owns goal alignment, decomposition and final synthesis, and it may hire further agents under approval.
- Every agent carries a role, a manager, an adapter, an instruction file, a heartbeat and an individual budget. Those six fields are the agent's whole contract.
- A role instruction states purpose, allowed inputs, the output schema, explicit exclusions and the escalation path — not a personality.
- Each agent gets a dedicated workspace: a separate local root for one role's files and evidence, isolated with git worktrees so changes are attributable.
- Secrets live in the runtime or in Paperclip's secret store with scoped injection at heartbeat time. They never appear in prompts, lab notes or Git.
- Skills are reusable agent capabilities imported from a path or URL into an org-wide library, then attached to specific agents — capability without retraining.
- Adapter readiness is proved by a marker run, not by the presence of configuration: an agent that cannot produce an observable artifact is not connected.
- Diagnose along a ladder — host, control plane, adapter, agent — and fix the earliest broken link before changing anything downstream.

Lab 2 — Create the BrightFunnel Company, Mission and CEO (Digital Marketing)

Maps to: LO2: Create a goal-aligned Paperclip company with a measurable mission, a hardened CEO agent and enforced company-level caps.

Simulated company: BrightFunnel Marketing (Digital Marketing).

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: You stand up BrightFunnel Marketing — a one-person digital marketing company — as a Paperclip company. You write a MEASURABLE mission (a target, a metric and a boundary), hire the Marketing Director as CEO, connect one trainer-approved agent runtime, and set the company budget cap BEFORE any agent is allowed to run. The order matters: a company that gains a runtime before it gains a cap can spend without a ceiling.

What you'll build

A BrightFunnel company in Paperclip with the exact synthetic mission, a hardened Marketing Director CEO on a verified adapter, a saved company budget cap, a dedicated workspace and a redacted configuration baseline in evidence/. (Tools: Paperclip onboarding wizard, Agent Configuration, one approved adapter runtime, text editor.)

Step-by-step

1. Open the Paperclip dashboard and start company creation. Use the exact synthetic company name so your evidence matches the marking guide.

```
http://localhost:3100  →  Companies → New Company
Name: BrightFunnel Marketing
```

2. Enter the MEASURABLE mission from the mock brief. A mission needs a target, a metric and a boundary — 'grow the funnel' is not verifiable, this is.

```
Increase qualified inbound leads by 30% in one quarter
while keeping blended cost per qualified lead under SGD 45.
```

3. Set the company-level budget cap BEFORE hiring. This is the outermost ring of the layered budget model and it must exist first.

```
Company → Settings → Budget
Monthly cap: SGD 40.00
```

4. Hire the CEO. The Marketing Director owns goal alignment, decomposition and final synthesis for BrightFunnel.

```
Agents → Hire → Role: Marketing Director
Reports to: (none — top of org chart)
```

5. Write the CEO instruction file. State PURPOSE, ALLOWED INPUTS, OUTPUT SCHEMA, EXCLUSIONS and ESCALATION — a role contract, not a personality.

```
workspace/marketing-director/INSTRUCTIONS.md
(copy the template from data/brightfunnel-ceo-instructions.md)
```

6. Attach a dedicated workspace so the CEO's files and evidence stay confined to one root and every change is attributable.

```
Agent → Workspace → Path: ./workspace/marketing-director
```

7. Select and verify the agent runtime adapter. Verification means Paperclip reports a successful connectivity check, not merely that a value was saved.

```
Agent → Runtime → (trainer-approved adapter)
Click Verify connection
```

8. Set the CEO's individual agent budget BELOW the company cap so the inner ring always binds first.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
Agent → Budget → Monthly cap: SGD 15.00
```

9. Import the BrightFunnel mock lead and channel data so the company has realistic synthetic material to reason over.

```
cp -r labs/lab-02-brightfunnel-marketing/data ./workspace/marketing-director/data
```

10. Record a redacted configuration baseline: company name, mission, CEO role, adapter NAME only, both budget caps and the workspace path. Never record keys.

```
evidence/lab02-brightfunnel-baseline.md
```

Test it

BrightFunnel exists with the exact mission text; the Marketing Director is the sole top-level agent with a verified adapter, a confined workspace and an instruction file; the agent cap (SGD 15) is below the company cap (SGD 40); and the baseline note contains no key, token or connection string.

Note: The lab folder labs/lab-02-* holds this lab's README and its synthetic data set. All data is fabricated for training — never substitute real company, market, financial or candidate data, and never record a credential in your evidence.

Lab 3 — Build the Meridian Reporting Tree and Validate Runtime Boundaries (Finance)

Maps to: LO2: Build a multi-agent reporting hierarchy with distinct, budgeted and confined workspaces, and prove each runtime boundary with a marker run.

Simulated company: Meridian Finance Desk (Finance).

Goal: You extend the pattern into Meridian Finance Desk and build a real three-agent org chart: a Finance Lead with an FP&A Analyst and a Treasury Analyst reporting to it. Each specialist receives its OWN workspace, its OWN instruction file and its OWN budget. You then prove every runtime boundary with a marker run — a deliberately trivial task whose only purpose is to produce an observable artifact in the correct workspace. Configuration that has never produced an artifact is not a connection.

What you'll build

A three-agent Meridian org chart with separate budgeted workspaces, role instruction files, clean Git baselines and one marker-run artifact per agent proving its runtime boundary. (Tools: Paperclip Agents, Org and Runs pages; approved adapter runtime; Git; Markdown editor.)

Step-by-step

1. Create the Meridian company with its measurable mission and a company budget cap, using the same order you proved in Lab 2.

```
Companies → New Company
Name: Meridian Finance Desk
Mission: Produce a board-ready monthly financial pack within 3 working days of
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
month end, with every figure traceable to a source ledger extract.
Company monthly cap: SGD 45.00
```

2. Hire the Finance Lead as CEO, then hire the two specialists AND set the reporting line explicitly. A flat list of agents is not an org chart.

```
Agents → Hire
  Finance Lead      (reports to: none)
  FP&A Analyst      (reports to: Finance Lead)
  Treasury Analyst  (reports to: Finance Lead)
```

3. Create one workspace root per agent. Separate roots are what make a later diff attributable to exactly one role.

```
mkdir -p workspace/{finance-lead,fpa-analyst,treasury-analyst}
```

4. Initialise Git in each workspace and commit an empty baseline, so every later change is a readable diff against a known-clean state.

```
for w in finance-lead fpa-analyst treasury-analyst; do \
  git -C workspace/$w init -q && \
  git -C workspace/$w commit -q --allow-empty -m 'baseline'; done
```

5. Write a role instruction file for each specialist stating purpose, allowed inputs, output schema, exclusions and escalation. The exclusions matter most.

```
workspace/fpa-analyst/INSTRUCTIONS.md
workspace/treasury-analyst/INSTRUCTIONS.md
(templates in data/ledger/meridian-role-instructions/)
```

6. Load the Meridian mock ledger extracts into a shared read-only data folder so both specialists reason over identical source figures.

```
cp -r labs/lab-03-meridian-finance/data/ledger ./data/ledger
```

7. Bind each agent to its own workspace path and its own adapter, then set an individual budget for each so the caps are layered company → agent.

```
Each agent → Workspace + Runtime + Budget
  Finance Lead     SGD 20.00
  FP&A Analyst     SGD 10.00
  Treasury Analyst SGD 10.00
```

8. Run a MARKER RUN for each agent: assign a trivial task whose only output is a single named file. This is how you prove the runtime boundary is real.

```
Issues → New
Title: Marker run — <agent>
Body: Create a file named marker.md in your workspace root containing only
      your agent role name and the current date. Do nothing else.
```

9. Inspect each run and confirm the artifact landed in the CORRECT workspace — and nowhere else. A marker in the wrong root is a boundary failure, not a success.

```
git -C workspace/fpa-analyst status --short
git -C workspace/treasury-analyst status --short
git -C workspace/finance-lead status --short
```

10. Record the org chart, the three budgets, the three marker-run IDs and the three artifact paths in your evidence note.

```
evidence/lab03-meridian-org.md
```

Test it

`paperclip → Org` shows a three-agent tree with both analysts reporting to the Finance Lead; each workspace contains exactly ONE marker.md attributable to its own agent;

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

every agent has an individual cap and the sum of agent caps does not exceed the company cap; and each marker run has a recorded run ID.

Note: The lab folder labs/lab-03-* holds this lab's README and its synthetic data set. All data is fabricated for training — never substitute real company, market, financial or candidate data, and never record a credential in your evidence.

Topic 03 — Developing Intelligent Multi-Agent Systems (25%)

Projects, goals and lineage · issue decomposition · assignment and hand-off · sequential specialists · verifying between disparate components

Key concepts

- Projects hold goals and issues and supply the context an agent reads at runtime, so tasks carry full goal ancestry — the agent sees why, not only what.
- Goal lineage flows downward: company mission → project goal → parent issue → child issue. An agent that cannot trace its lineage is working without direction.
- Decompose into a parent synthesis issue plus specialist child issues. Create every issue unassigned first, so creation cannot accidentally wake an agent.
- Assignment is the trigger. Assign one child at a time to keep runs sequential, observable and individually attributable.
- An acceptance criterion must name the evidence: which artifact, at which path, containing which fields — never 'the research is done'.
- A hand-off is accepted only when the receiving agent can consume the artifact without asking the producer a question.
- Verification between disparate components means tracing one thread across issue ID, run log, comment, workspace diff and cost entry, and finding them consistent.
- Status is not evidence. An issue marked Done with no linked artifact and no diff is an unverified claim.
- Explicit limitations are part of a good deliverable — an agent that reports what it could not establish is more useful than one that quietly guesses.
- One event should not create many outcomes: choose a single trigger for the first controlled run and keep the work idempotent.
- Bound every wait. An unbounded wait for a heartbeat hides a failed adapter behind what looks like patience.
- The workspace diff is the strongest evidence you have — read what actually changed on disk before accepting any agent's summary of it.

Lab 4 — Translate the Ledgerly Mission into Assigned Issues and Hand-Offs (Accounting)

Maps to: LO3: Decompose a mission into a parent and specialist child issues, run the specialists sequentially and verify each hand-off against acceptance evidence.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Simulated company: Ledgerly Accounting (Accounting).

Goal: You turn the Ledgerly Accounting mission into actual work. You create a budgeted month-end close project, then a PARENT synthesis issue and two specialist CHILD issues — all created UNASSIGNED, so that creating work cannot accidentally wake an agent. Only after every acceptance criterion names its evidence do you assign one child at a time. This is the core discipline of the topic: assignment is the trigger, and sequential assignment is what makes each run individually verifiable.

What you'll build

A capped Ledgerly close project with an unassigned parent issue, two sequentially completed specialist children, clean workspace manifests, and an evidence record linking issue IDs, run IDs, comments, statuses and artifact paths. (Tools: Paperclip Projects, Budgets, Issues, Agents and Runs; approved adapter runtime; Git; Markdown workspaces.)

Step-by-step

1. Create the Ledgerly company, hire the Practice Manager as CEO plus a Reconciliation Specialist and a Compliance Reviewer, and set layered caps — the pattern you proved in Labs 2 and 3.

```
Companies → New Company: Ledgerly Accounting
Mission: Close the books each month with zero unexplained variances above SGD 500
and a complete reconciliation trail for every control account.
Company cap SGD 45 · CEO SGD 20 · each specialist SGD 10
```

2. Create the project and give it its OWN budget cap. The project ring sits between the company and the agent, and it is what stops one project consuming the company.

```
Projects → New
Name: October Month-End Close
Goal: Close October with every control account reconciled and evidenced.
Project cap: SGD 25.00
```

3. Load the Ledgerly mock trial balance and bank statement extracts into a shared read-only data folder.

```
cp -r labs/lab-04-ledgerly-accounting/data/close ./data/close
```

4. Create the PARENT synthesis issue UNASSIGNED. Its acceptance criterion names the artifact, its path and the fields it must contain.

```
Issues → New  (Assignee: leave EMPTY)
Title: Produce the October close pack
Accept: workspace/practice-manager/close-pack.md exists and contains, for every
control account: account, ledger balance, statement balance, variance, explanation.
```

5. Create the two CHILD issues, both unassigned, each parented to the synthesis issue so goal lineage is explicit and traceable.

```
Child 1 → Reconcile control accounts   (parent: Produce the October close pack)
Child 2 → Review compliance exceptions (parent: Produce the October close pack)
```

6. Write an acceptance criterion for each child that names the EVIDENCE, not the activity. 'Reconciliation done' is unverifiable; a named file with named fields is.

```
Child 1 accept: workspace/reconciliation-specialist/reconciliation.md lists every
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
  control account with ledger balance, statement balance and variance.
Child 2 accept: workspace/compliance-reviewer/exceptions.md lists each variance
  above SGD 500 with a stated cause and a recommended treatment.
```

7. Assign CHILD 1 ONLY. Assignment is the trigger — this is the moment the Reconciliation Specialist wakes.

```
Child 1 → Assignee: Reconciliation Specialist
```

8. Wait for the heartbeat with a BOUND. Inspect the run log rather than the issue status — an unbounded wait hides a failed adapter behind apparent patience.

```
Runs → (latest)   ·   bound the wait to 10 minutes
git -C workspace/reconciliation-specialist status --short
```

9. Accept child 1 ONLY if the named artifact exists with the named fields. Record the run ID and the artifact path in a comment on the issue, then close it.

```
Issue → Comment: run <run-id> produced reconciliation.md (<n> accounts)
Issue → Status: Done
```

10. Now assign CHILD 2. Running the specialists sequentially is what keeps each run individually attributable and each hand-off inspectable.

```
Child 2 → Assignee: Compliance Reviewer
```

11. Verify the HAND-OFF: confirm the Compliance Reviewer consumed child 1's artifact without needing to ask the producer a question.

```
grep -c 'SGD' workspace/compliance-reviewer/exceptions.md
git -C workspace/compliance-reviewer diff --stat HEAD
```

12. Record the evidence chain — for each child: issue ID, run ID, artifact path, comment and final status — in one table.

```
evidence/lab04-ledgerly-handoffs.md
```

Test it

The Ledgerly project shows a saved cap above actual spend; the parent issue is still open and unassigned; both children are Done with a linked artifact and a recorded run ID; the two specialists ran SEQUENTIALLY (non-overlapping run timestamps); and the exceptions file demonstrably references the reconciliation figures.

Note: The lab folder labs/lab-04-* holds this lab's README and its synthetic data set. All data is fabricated for training — never substitute real company, market, financial or candidate data, and never record a credential in your evidence.

Lab 5 — Verify NorthPoint Routines and Layered Budget Controls (Stock Trading Research)

Maps to: LO3: Test a scheduled routine and verify that every budget layer holds, using one bounded manual run before automation is allowed to own the schedule.

Simulated company: NorthPoint Trading Research (Stock Trading — research only).

Goal: NorthPoint Trading Research needs a daily pre-market brief — which makes it the right place to learn routines. You create the readiness routine PAUSED, add an explicit cron trigger with an IANA timezone, prove the routine once by hand, and leave it paused

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

afterwards. You then verify all three budget layers against actual recorded spend. The rule the lab teaches: automation earns the schedule only after a human has watched it succeed once. This is research only — the company never places or recommends a trade.

What you'll build

A paused NorthPoint readiness routine with a saved 07:00 Asia/Singapore trigger, exactly one reviewed manual run, and a control table proving company, project and agent budget layers all remain above recorded spend. (Tools: Paperclip Routines, Costs and Budgets, Agent Budget tabs, Project settings, Activity.)

Step-by-step

1. Create the NorthPoint company with a Research Director CEO, a Market Data Analyst and a Risk Reviewer, with layered caps. Note the research-only boundary in the mission — the company must never recommend a trade.

```
Companies → New Company: NorthPoint Trading Research
Mission: Deliver a daily pre-market research brief on a fixed synthetic watchlist
by 07:30 SGT, with every claim sourced and every limitation stated.
Company cap SGD 40 · CEO SGD 15 · each analyst SGD 10
```

2. Load the synthetic watchlist and price history. This data is fabricated for training and must never be presented as market data.

```
cp -r labs/lab-05-northpoint-trading/data/market ./data/market
```

3. Create the routine and set it PAUSED at creation. A routine that is live before it has ever succeeded is an unreviewed scheduled spend.

```
Routines → New
Name: Daily pre-market readiness check
State: PAUSED
```

4. Add the schedule trigger EXPLICITLY, with the cron expression and the IANA timezone. An implicit timezone is a production incident waiting to happen.

```
Trigger → Schedule
Cron: 0 7 * * *
Timezone: Asia/Singapore
```

5. Write what the routine actually does, naming the artifact it must produce so the run is verifiable rather than merely successful.

```
Body: Produce workspace/market-data-analyst/premarket-brief.md summarising each
watchlist symbol from ./data/market ONLY. State sources and limitations.
Do not recommend any trade.
```

6. Run the routine ONCE manually while it is still paused, and bound the wait. This is the human review that earns the schedule.

```
Routine → Run now   ·   bound the wait to 10 minutes
```

7. Inspect the generated issue and the run — confirm the routine created work correctly and that the named artifact exists.

```
Issues → (routine-generated issue)
Runs → (latest)
cat workspace/market-data-analyst/premarket-brief.md | head -30
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

8. Read the recorded COST for that run. This actual figure is what you will compare every budget layer against.

```
Costs → filter by agent: Market Data Analyst
Record the run cost
```

9. Confirm the routine is STILL paused after the manual run. Leaving it live is the most common mistake in this lab.

```
Routines → Daily pre-market readiness check → State: PAUSED
```

10. Verify every budget layer against actual spend and record the three-row control table: company, project and agent — each cap, each spend, each headroom.

```
evidence/lab05-northpoint-budgets.md
| Layer | Cap | Spend | Headroom |
```

11. Confirm the audit trail: the activity log must show the routine creation, the trigger addition, the manual run and the cost event as separate entries.

```
Activity → filter: NorthPoint Trading Research
```

Test it

The routine carries an explicit `0 7 * * *` Asia/Singapore trigger, has exactly ONE successful reviewed manual run, and finishes PAUSED; the brief exists and cites only the synthetic data with limitations stated and no trade recommendation; and the control table shows all three budget layers above recorded spend.

Note: The lab folder labs/lab-05-* holds this lab's README and its synthetic data set. All data is fabricated for training — never substitute real company, market, financial or candidate data, and never record a credential in your evidence.

Topic 04 — Deploying and Managing AI Agents (25%)

Routines and schedules · layered budgets · approvals and audit · operator intervention · deployment, portability and troubleshooting

Key concepts

- A routine is recurring work triggered by cron, webhook or API. Create it paused, prove it once by hand, and only then let the schedule own it.
- A schedule trigger is explicit — `30 16 * * *` with an IANA timezone such as Asia/Singapore. An implicit timezone is a production incident waiting to happen.
- Budgets are layered: company, project, agent. Enforcement happens at checkout, so a runaway loop stops at the ceiling instead of after you read the invoice.
- Treat the budget as a two-stage circuit breaker — warn, then stop — and record the baseline spend and expected run frequency before you rely on it.
- Cost is tracked at token level by provider, model, agent and project, which is what makes the per-agent cap meaningful rather than notional.
- Approvals are the board's authority made explicit: the agent proposes a hire, a strategy or a risky action, and a human decides with a recorded rationale.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- The activity log is a durable record of mutations, heartbeats, cost events and approvals — the audit loop closes when the whole operation reconstructs from it.
- Use the least irreversible control first: inspect, then pause, then reassign, then terminate. Retirement is the last rung of the ladder, not the first.
- Deployment options run from local loopback through LAN or tailnet binding to a hosted container with an external PostgreSQL and a persistent volume.
- A production deployment needs the signing secret for tool approvals, the allowed hostnames list and a health check on /api/health.
- Portability means an entire company exports and imports with secrets scrubbed and collisions handled — the org chart is data, not a hand-built configuration.
- Troubleshooting is a discipline: reproduce, isolate the layer, change one setting, re-verify with evidence, and record what the fix actually proved.

Lab 6 — Run the Governed TalentBridge Operation and Close the Audit Loop (HR)

Maps to: LO4: Resolve issues and modify a running agent company through approvals, the operator intervention ladder and a reconstructed audit trail.

Simulated company: TalentBridge People Ops (HR — sourcing, interview, policy, onboarding).

Goal: TalentBridge People Ops is the highest-stakes simulation in the course: hiring decisions affect real people, so every control you have learned is now mandatory rather than advisable. You complete the sourcing-to-onboarding operation, review the final synthesis against a workspace DIFF before the board closes it, process a bounded hire approval with a recorded rationale, practise the intervention ladder on a disposable role only, and finally reconstruct the entire operation from the activity log. The audit loop closes when the log alone tells the whole story.

What you'll build

A diff-reviewed TalentBridge hiring brief, a separately confined reviewer with a recorded approval rationale, a lifecycle intervention record, and a mission-to-evidence audit map reconstructed from the activity log alone. (Tools: Paperclip Issues, Agents, Runs, Approvals, Activity, Costs and local Markdown artifacts.)

Step-by-step

1. Create the TalentBridge company with a People Operations Lead as CEO, a Talent Sourcer and a Policy & Onboarding Specialist, with layered caps.

```
Companies → New Company: TalentBridge People Ops
Mission: Fill each open role within 30 days with a fair, documented and auditable
process from sourcing through to day-one onboarding.
Company cap SGD 45 · CEO SGD 20 · each specialist SGD 10
```

2. Load the synthetic candidate pipeline, the interview rubric and the onboarding checklist. Every candidate record is fabricated — never use real applicant data.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cp -r labs/lab-06-talentbridge-hr/data/hiring ./data/hiring
```

3. Build the issue tree as in Lab 4: an unassigned parent plus two children, then run the two specialists sequentially and accept each on named evidence.

```
Parent (unassigned): Produce the Support Engineer hiring brief
  Child 1 → Shortlist candidates against the rubric  (Talent Sourcer)
  Child 2 → Draft policy-compliant onboarding plan   (Policy & Onboarding)
```

4. Only after BOTH children are accepted, assign the parent so the CEO synthesises. Assigning the parent early is what produces a synthesis of incomplete work.

```
Parent → Assignee: People Operations Lead   ·   bound the wait to 10 minutes
```

5. Review the final synthesis against the workspace DIFF, not against the agent's own summary. The diff is the strongest evidence you have.

```
git -C workspace/people-ops-lead diff HEAD --stat
git -C workspace/people-ops-lead diff HEAD -- hiring-brief.md
```

6. Confirm fairness controls in the brief: every shortlisting reason must trace to the published rubric, and no protected characteristic may appear as a criterion.

```
grep -nEi 'age|gender|race|religion|marital|nationality' \
  workspace/people-ops-lead/hiring-brief.md   # expect: no criteria hits
```

7. Close the parent only after board sign-off, recording WHO approved and on what evidence. A close without a rationale is an unaudited decision.

```
Issue → Comment: approved by <name> on diff <sha>, rubric-traceable
Issue → Status: Done
```

8. Process a bounded HIRE APPROVAL: let the CEO propose one additional agent, then decide it yourself and record the rationale in the approval.

```
Approvals → (pending hire proposal)
Decide: Approve or Reject   ·   Rationale: <why, in one sentence>
```

9. Confine the newly approved reviewer to its OWN workspace and its own budget before it ever runs — a new hire inherits nothing by default.

```
Agent → Workspace: ./workspace/interview-reviewer
Agent → Budget: SGD 5.00
```

10. Practise the operator INTERVENTION LADDER on the disposable reviewer role only, in order: inspect → pause → reassign → terminate. Use the least irreversible control that resolves the situation.

```
Agent → Inspect runs and costs
Agent → Pause
Issue → Reassign to People Operations Lead
Agent → Terminate  (disposable role ONLY)
```

11. Reconstruct the whole operation from the ACTIVITY LOG alone — company creation, hires, issues, runs, costs, the approval and the interventions, in order.

```
Activity → filter: TalentBridge People Ops → export or transcribe in order
```

12. Write the mission-to-evidence audit map: one row per mission claim, linking the issue, the run, the artifact, the cost entry and the approval that evidences it.

```
evidence/lab06-talentbridge-audit-map.md
| Mission claim | Issue | Run | Artifact | Cost | Approval |
```

Test it

The hiring brief is the only later People Ops Lead workspace change and its diff passes the rubric-traceability and fairness checks; the approval carries a recorded rationale; the

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

intervention ladder was applied to the disposable role only and in order; and the audit map reconstructs every mission claim from activity-log entries with no gap.

Note: The lab folder labs/lab-06-* holds this lab's README and its synthetic data set. All data is fabricated for training — never substitute real company, market, financial or candidate data, and never record a credential in your evidence.

Operating Reference — Beyond the Labs

These cross-cutting topics are assessed throughout the course but do not each have a dedicated lab. Read this section alongside the labs so you can answer the knowledge questions and run a real Paperclip company safely afterwards.

Choosing the deployment mode

- Loopback (the default) — trusted local only, the least exposed and the right choice for this course.
- `--bind lan` — reachable from your network; authentication becomes mandatory, not optional.
- `--bind tailnet` — reachable over a private mesh; again authentication is required.
- Hosted / container — an external PostgreSQL, a persistent volume for instance config and credentials, PAPERCLIP_TOOL_ACTION_SIGNING_SECRET for tool approvals, and an allowed-hostnames list.
- /api/health is the readiness probe at every one of these levels.

Budget layering in practice

- Set the COMPANY cap before you hire anyone; set the AGENT cap below it; add a PROJECT cap where one project must not consume the company.
- Caps are enforced at CHECKOUT, not at billing — that is what makes unattended operation safe.
- Record a baseline (current spend, expected run frequency) before you rely on a cap.
- Treat it as a two-stage circuit breaker: warn, then stop. A cap you would be unhappy to actually pay is set too high.
- Cost is tracked per token by provider, model, agent and project — read the actual figure before adjusting any cap.

Routines and schedule safety

- Create every routine PAUSED. A routine that is live before it has ever succeeded is unreviewed scheduled spend.
- State the trigger explicitly — a cron expression AND an IANA timezone (e.g. `30 16 * * *`, Asia/Singapore). An implicit timezone is a production incident waiting to happen.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Prove it with exactly one bounded manual run, reviewed by a human, then leave it paused until you have read the generated issue, the run and the cost.
- Name the artifact the routine must produce, or the run is successful but unverifiable.

Governance, approvals and intervention

- Approvals make human authority explicit: the agent proposes, the action HOLDS, a human decides with a recorded rationale. A close without a rationale is an unaudited decision.
- The activity log is durable — mutations, heartbeats, cost events and approvals. The audit loop closes when the whole operation reconstructs from the log alone.
- Use the least irreversible control first: inspect → pause → reassign → terminate. Terminate is the last rung and is not reversible.
- An unexplained gap in the activity log usually means work was done outside a run. Record it honestly — the gap is itself an audit finding.

Working safely with synthetic data

- All six company data sets in this course are fabricated. The NorthPoint market data is not market data and must never inform a real trading decision.
- The TalentBridge candidate file deliberately records NO protected characteristics — no age, gender, race, religion, marital status or nationality. Keep it that way in any real hiring workflow.
- Keep credentials in the runtime or the Paperclip secret store with scoped injection at heartbeat time — never in prompts, notes or Git.

Course Preparation and Practice

- First pass: complete every lab in order, reading each lab folder's README and its data/ set.
- Second pass: rebuild one company from scratch without the guide, until the control sequence — cap, hire, confine, assign, verify, audit — is automatic.
- Review the 'Test it' criterion for any lab you found hard; it states exactly what the assessor will look for.
- Practise writing acceptance criteria that name an artifact, its path and its required fields — this is the skill most learners under-practise.
- Sharpen your readiness with the Tertiary Infotech practice exam: https://exams.tertiaryinfotech.com
- Then apply it: pick ONE real internal workflow whose inputs and outputs are already observable, cap it low, and audit it before you scale.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Glossary

- **Control plane —** The layer that coordinates agents, goals, work, cost and governance. Paperclip is a control plane; it does not perform the reasoning itself.
- **Adapter / runtime —** The agent that actually reasons and uses tools — Claude Code, Codex, Cursor, Gemini CLI, a bash process or an HTTP bot. 'If it can receive a heartbeat, it's hired.'
- **Company —** The boundary for one mission, its human members, AI employees, projects, issues, budget and activity history. One deployment holds many companies.
- **Mission —** The measurable outcome that defines why a company exists — it needs a target, a metric and a boundary, or it cannot be verified.
- **CEO —** The first hire and top of the org chart; owns goal alignment, decomposition and final synthesis, and may hire further agents under approval.
- **Org chart —** The explicit reporting structure — who reports to whom, with what authority, budget and workspace.
- **Agent —** A work-performing role defined by six fields: role, manager, adapter, instructions, heartbeat and budget.
- **Role instruction —** A contract stating purpose, allowed inputs, output schema, exclusions and escalation path — not a personality.
- **Workspace —** A dedicated local root for one role's files and evidence, isolated with git worktrees so every change is attributable.
- **Heartbeat —** A bounded execution burst triggered by a timer, an assignment, an on-demand wake or a routine. Between heartbeats the agent is dormant.
- **Atomic checkout —** The execution lock that stops two heartbeats double-working the same issue or racing past a budget ceiling.
- **Issue —** A unit of work carrying full goal ancestry, so the agent sees why it exists and not only what to do.
- **Goal lineage —** The chain company mission → project goal → parent issue → child issue that gives an agent its direction.
- **Acceptance criterion —** A statement naming the required evidence — the artifact, its path and its fields. Never 'the work is done'.
- **Hand-off —** Passing an artifact between agents. It is accepted only when the receiver can consume it without asking the producer a question.
- **Run —** One agent execution instance, with a structured log, a cost entry and an observable result.
- **Routine —** Recurring work triggered by cron, webhook or API. Create it paused, prove it once by hand, then let the schedule own it.
- **Budget layering —** Company, project and agent caps. The innermost cap that binds fires first, and enforcement happens at checkout.
- **Approval —** A governance gate: the agent proposes a hire, strategy or risky action; the action holds; a human decides with a recorded rationale.
- **Activity log —** The durable record of mutations, heartbeats, cost events and approvals — the basis of the audit loop.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **Intervention ladder —** Inspect → pause → reassign → terminate. Always use the least irreversible control that resolves the situation.
- **Skill —** A reusable agent capability imported into an org-wide library and attached to specific agents — capability without retraining.
- **Portability —** Export/import of a whole company with secrets scrubbed and identifier collisions re-mapped.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg